

سوال اول

@WebMvcTest برای تست های مرتبط با MVC طراحی شده است. دیزاین MVC برنامه را به سه بخش Model-View-Controller تقسیم می کند تا کدهای هر بخش جدا باشند. @WebMvcTest روی لایه وب تمرکز دارد و تست کردن بخش controller ها را آسان می کند. در نتیجه برای اینکه تست سریع تر انجام شود فقط لایه وب را لود می کند و service ها را با @MockBean ماک می کند و از service های واقعی استفاده نمی کند.

از آن طرف @SpringBootTest تمام برنامه را لود می کند، که شامل @Component ها، ارتباطات با پایگاه داده و @Service ها می شود. در نتیجه برای integration testing و system testing مناسب است. یعنی شرایطی شبیه محیط production را ایجاد می کند. و این باعث می شود که لود، هزینه و زمان بیشتری داشته باشد و وقتی ما می خواهیم صرفاً controller ها را تست کنیم نیازی نیست چنین هزینه ای بپردازیم.

سوال دوم

(الف)

#	a	b	c	p
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	T
5	F	T	T	T
6	F	T	F	T
7	F	F	T	F
8	F	F	F	T

اگر چه با توجه به جدول بالا می‌توانیم شرط major clause بودن را بررسی کنیم اما در ادامه به روش تئوری نیز این شرط را بدست می‌آوریم:

clause a:

$$\begin{aligned} P_{a=false} &= (T \wedge b) \vee (b \wedge c) \vee (\neg b \wedge \neg c) \\ &= b \vee \neg c \end{aligned}$$

$$\begin{aligned} P_{a=true} &= (F \wedge b) \vee (b \wedge c) \vee (\neg b \wedge \neg c) \\ &= (b \wedge c) \vee (\neg b \wedge \neg c) \end{aligned}$$

$$\begin{aligned} P_a &= P_{a=false} \oplus P_{a=true} \\ &= (b \vee \neg c) \oplus ((b \wedge c) \vee (\neg b \wedge \neg c)) \\ &= (b \vee \neg c) \oplus b \oplus \neg c \\ &= b \wedge \neg c \end{aligned}$$

clause b:

$$\begin{aligned} P_{b=false} &= (\neg a \wedge F) \vee (F \wedge c) \vee (T \wedge \neg c) \\ &= \neg c \end{aligned}$$

$$\begin{aligned} P_{b=true} &= (\neg a \wedge T) \vee (T \wedge c) \vee (F \wedge \neg c) \\ &= \neg a \vee c \end{aligned}$$

$$\begin{aligned} P_b &= P_{b=false} \oplus P_{b=true} \\ &= (\neg c) \oplus (\neg a \vee c) \\ &= c \vee a \end{aligned}$$

clause c:

$$\begin{aligned}P_{c=false} &= (\neg a \wedge b) \vee (b \wedge F) \vee (\neg b \wedge T) \\&= (\neg a \wedge b) \vee \neg b \\&= (\neg a \vee \neg b) \wedge (b \vee \neg b) \\&= \neg a \vee \neg b\end{aligned}$$

$$\begin{aligned}P_{c=true} &= (\neg a \wedge b) \vee (b \wedge T) \vee (\neg b \wedge F) \\&= (\neg a \wedge b) \vee b \\&= b\end{aligned}$$

$$\begin{aligned}P_c &= P_{c=false} \oplus P_{c=true} \\&= (\neg a \vee \neg b) \oplus (b) \\&= ((\neg a \vee \neg b) \wedge \neg b) \vee (a \wedge b \wedge b \wedge b) \\&= (\neg b) \vee (a \wedge b) \\&= (\neg b \vee a) \wedge (\neg b \vee b) \\&= \neg b \vee a\end{aligned}$$

برای راحتی محاسبات بخش‌های بعدی سطرهایی که هر clause در آن major clause است به همراه نتیجه آن عبارت در آن سطر را می‌نویسیم:

clause a: [(2→F), (6→T)]

clause b: [(1→T), (2→F), (3→F), (4→T), (5→T), (7→F)]

clause c: [(1→T), (2→F), (3→F), (4→T), (7→F), (8→T)]

(ب)

clause a: [(2, 6)]

clause b: [(1, 3), (1, 7), (2, 4), (5, 3), (5, 7)]

clause c: [(1, 2), (3, 4), (3, 8), (7, 4), (7, 8)]

(پ)

clause a: [(2, 6)]

clause b: [(1, 3), (2, 4), (5, 7)]

clause c: [(1, 2), (3, 4), (7, 8)]

جفت‌های مورد (ب) زیر مجموعه جفت‌های مورد (پ) نمی‌باشند زیرا زمانی که minor clause تغییر می‌کند اما همچنان با هر دو عضو یک جفت، می‌توان هم به حالت $P = \text{false}$ و هم $P = \text{true}$ دست‌یافت ما CACC

را برآورده می‌کنیم، در حالی که چنین حالتی به دلیل یکسان نبودن minor clause در دو عضو جفت RACC را برآورده نخواهد کرد در نتیجه چنین اعضایی در مورد (ب) دیده می‌شود اما در مورد (پ) وجود ندارند.

(ت)

خیر. در این عبارت clause coverage لزوماً predicate coverage را نتیجه نمی‌دهد. مثلاً TR زیر را در نظر بگیرید که در آن همهی clause-ها هر دو حالت true و false را گرفته‌اند یعنی clause coverage برآورده شده است اما در هر دو مورد نتیجه P برابر true خواهد بود یعنی predicate coverage برآورده نشده است.

$$TR = \{(1, 8)\}$$

سوال سوم

بلاک‌های هر پارامتر را به صورت زیر در نظر می‌گیریم:

price .1

- price < 0 :A1
- price = 0 :A2
- price > 0 :A3

discountRate .2

- discountRate < 0 :B1
- discountRate = 0 :B2
- discountRate < 1 و discountRate > 0 :B3
- discountRate = 1 :B4
- discountRate > 1 :B5

minPurchase .3

- minPurchase < 0 :C1
- minPurchase = 0 :C2
- minPurchase > 0 :C3

برای اینکه PWC را برآورده کنیم می‌توانیم تست‌های زیر را انتخاب کنیم:

A1, B1, C1	A1, B2, C3	A1, B3, C2	A1, B4, C1	A1, B5, C3
A2, B1, C2	A2, B2, C1	A2, B3, C3	A2, B4, C2	A2, B5, C1
A3, B1, C3	A3, B2, C2	A3, B3, C1	A3, B4, C3	A3, B5, C2

یک نمونه از تست‌کیس آن در ادامه نشان داده شده است:

```

@Test
void testCalculateDiscountedPrice() {
    final String INVALID_INPUT = "Invalid input";

    double[] A = {-1, 0, 1};
    double[] B = {-1, 0, 0.5, 1, 2};
    double[] C = {-1, 0, 1};

    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[0], B[0], C[0]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[0], B[1], C[2]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[0], B[2], C[1]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[0], B[3], C[0]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[0], B[4], C[2]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[1], B[0], C[1]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[1], B[1], C[0]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[1], B[2], C[2]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[1], B[3], C[1]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[1], B[4], C[0]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[2], B[0], C[2]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[2], B[1], C[1]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[2], B[2], C[0]));
    assertEquals("0.0", calculateDiscountedPrice(A[2], B[3], C[2]));
    assertEquals(INVALID_INPUT, calculateDiscountedPrice(A[2], B[4], C[1]));
}

```

در این آزمون برای سادگی و تمرکز بر مفاهیم بیان شده در سوال، همگی تست‌ها در یک تابع نوشته شده است و قوانین تمیزی کد در آن رعایت نشده است. همچنین نام‌گذاری برای مطابقت با روش تئوری چنین انتخاب شده است و در عمل این نوع نام‌گذاری مناسب نیست.